

VULNERABILITY ASSESSMENT REPORT

OWASP Juice Shop

Future Interns Cyber Security Internship — Task 1

Target	http://127.0.0.1:3000
Assessment Type	Passive / Read-only
Environment	Locally hosted training application (Docker)
Assessment Date	4 September 2026
Report Date	5 September 2026
Classification	Internship Portfolio — Confidential

Prepared as part of the Future Interns Cyber Security Internship Program
Passive, non-destructive assessment — no exploitation performed

1. Executive Summary

This report presents the results of a vulnerability assessment conducted against a locally hosted instance of the OWASP Juice Shop training application, performed as Task 1 of the Future Interns Cyber Security Internship program. The objective of the assessment was to identify common web security weaknesses, classify them using business-friendly risk levels, and provide practical remediation guidance.

The assessment combined service identification with Nmap, a passive (Standard Mode) OWASP ZAP scan, direct HTTP response-header inspection, and manual review of browser-side storage and cookies using Firefox Developer Tools. Testing was intentionally limited to passive, read-only, and non-destructive techniques. No login bypass, brute-force activity, denial-of-service testing, credential attacks, or exploitation were performed.

OWASP ZAP identified 13 alert categories across 49 observed endpoints, comprising 0 High, 4 Medium, 4 Low, and 5 Informational alerts. Following manual review, seven findings (F-01 through F-07) were selected for detailed discussion because they represent concrete, actionable configuration weaknesses; the remaining five ZAP alerts were determined to be informational observations rather than security findings and are discussed separately in Section 10.

The most notable observations were the absence of a Content Security Policy header, a permissive (wildcard) Cross-Origin Resource Sharing configuration, authentication-related data stored in browser Local Storage, disclosure of private/internal IP addresses in an administrative API response, and endpoint-specific gaps in clickjacking and MIME-sniffing protection. None of these findings were exploited as part of this assessment; they are presented as configuration-hardening opportunities appropriate to the passive scope of the engagement.

Full risk classifications, technical detail, and remediation guidance for each finding are provided in Sections 8, 9, and 12 respectively.

2. Assessment Objective

The objective of this engagement, as defined by the Future Interns Cyber Security Internship (Task 1 — Vulnerability Assessment Report for a Live Website), was to:

- Identify common web application security weaknesses in the target application.
- Classify identified weaknesses using business-friendly risk levels (Low / Medium / High).
- Provide practical, actionable remediation recommendations for each finding.
- Produce a professional vulnerability assessment report containing supporting evidence and tool output.

To keep the assessment controlled, authorized, and safe to publish as an internship portfolio artifact, the target was intentionally deployed locally rather than tested against a third-party production system.

3. Scope

3.1 In Scope

- Locally hosted OWASP Juice Shop application at <http://127.0.0.1:3000>

- TCP port 3000 service identification
- HTTP response header analysis
- Passive OWASP ZAP observations (Standard Mode)
- Browser Network tab inspection
- Browser Local Storage and Session Storage review
- Browser cookie inspection
- Security configuration observations
- Information-disclosure observations

3.2 Out of Scope

- Login bypass
- Exploitation of identified weaknesses
- Brute-force attacks
- Denial-of-service testing
- Destructive requests of any kind
- Credential attacks
- Testing of unrelated systems or unrelated public websites

The intended methodology throughout was passive and non-destructive.

4. Environment

The assessment was conducted from a Kali Linux workstation against a Docker-deployed instance of OWASP Juice Shop, accessed locally through Firefox. Relevant environment and tooling versions are listed below.

Operating System	Kali Linux Rolling 2026.1
Kernel	6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64
Nmap	7.99
OWASP ZAP	2.17.0-0kali1
Java	OpenJDK 25.0.4
Firefox	140.13.0esr
Docker	28.5.2+dfsg4
Node.js	v24.19.0
npm	11.16.0
Git	2.53.0

Application	OWASP Juice Shop
Target URL	http://127.0.0.1:3000
Deployment	Docker (locally hosted training instance)
Access Method	Firefox, direct local connection

5. Methodology

The assessment followed a passive, read-only methodology consisting of four stages:

- Service identification — an Nmap version-detection scan against the target port to confirm the exposed service.
- HTTP response header review — direct inspection of the application's HTTP headers using curl, to independently verify header-related ZAP findings.
- Passive web application scanning — an OWASP ZAP scan of the application operated in Standard Mode, relying on passive analysis of proxied traffic rather than active attack payloads.
- Browser-side review — manual inspection of Local Storage, Session Storage, and cookies using Firefox Developer Tools, to independently confirm client-side storage behavior observed by ZAP.

An automated/Attack Mode ZAP scan was briefly and unintentionally started during environment setup. This scan was not used as the basis for any finding in this report; all findings and statistics presented here are drawn exclusively from the passive, Standard Mode assessment described above.

A full statement of testing ethics and authorization is provided in Section 15.

6. Tools Used

Tool	Version	Purpose
Nmap	7.99	TCP port and service-version identification
OWASP ZAP	2.17.0-0kali1	Passive (Standard Mode) web application scanning
curl	System default (Kali Rolling 2026.1)	Direct HTTP response-header verification
Firefox Developer Tools	140.13.0esr	Local Storage, Session Storage, and cookie inspection

7. Assessment Results

7.1 Nmap Service Identification

An Nmap version-detection scan was run against the target port:

```
nmap --privileged -sV -p 3000 -oN evidence/nmap/01-basic-service-scan.txt 127.0.0.1
```

```
PORT      STATE SERVICE VERSION
3000/tcp  open  ppp?
1 service unrecognized despite returning data.
```

Nmap was unable to positively fingerprint the service and labeled it "ppp?". This does not indicate that the Point-to-Point Protocol is in use; it reflects only that the response did not match a signature in Nmap's service-probe database with certainty. The captured response fingerprint clearly showed a standard HTTP response, including headers and HTML markup consistent with the OWASP Juice Shop application (see Section 13, Figure 1). TCP port 3000 is confirmed open and serving HTTP content; the "ppp?" label itself is not a vulnerability.

7.2 HTTP Response Header Review

A direct HTTP request was issued against the application root to independently verify header behavior:

```
curl -I http://127.0.0.1:3000
```

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Fri, 04 Sep 2026 12:34:57 GMT
ETag: W/"24b1-1a06c6a4001"
Content-Type: text/html; charset=UTF-8
Content-Length: 9393
Vary: Accept-Encoding
Date: Fri, 04 Sep 2026 12:35:23 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

This confirms that the application's homepage response does include both X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN. Several ZAP alerts relating to these headers are endpoint-specific rather

than site-wide; this distinction is preserved throughout Sections 9 and 10, and this report does not characterize the application as entirely missing these protections.

7.3 OWASP ZAP Passive Assessment Summary

OWASP ZAP 2.17.0 was operated in Standard Mode against `http://127.0.0.1:3000`. The scan observed 49 distinct endpoints and produced the following traffic and severity profile:

Metric	Value
Observed endpoints	49
HTTP methods	91% GET, 8% POST
Response codes	78% 2xx, 17% 3xx, 2% 4xx, 1% 1xx
Slow responses	12%
High-risk alerts	0
Medium-risk alerts	4
Low-risk alerts	4
Informational alerts	5
Total alert categories	13

OWASP ZAP identified 13 alert categories comprising 4 Medium, 4 Low, and 5 Informational alerts. This figure refers to distinct alert categories reported by the scanner, not to 13 individually confirmed vulnerabilities; several categories represent the same underlying observation recurring across multiple endpoints (for example, three instances of missing X-Content-Type-Options). Each category was reviewed individually and is discussed in Sections 9 and 10.

8. Risk Summary

The table below summarizes the seven findings selected for detailed discussion in this report, drawn from the 13 ZAP alert categories. Findings marked with an asterisk (*) are contextual and require further validation before being treated as confirmed, exploitable vulnerabilities; see Section 14 (Limitations) for detail.

ID	Finding	Risk	Source
F-01	Content Security Policy (CSP) Header Not Set	Medium	OWASP ZAP
F-02	Permissive Cross-Origin Resource Sharing (CORS)	Medium	OWASP ZAP + HTTP headers
F-03	Missing anti-clickjacking header on Socket.IO endpoint	Medium*	OWASP ZAP + HTTP

ID	Finding	Risk	Source
			inspection
F-04	Session identifier in Socket.IO URL	Medium*	OWASP ZAP
F-05	Authentication-related data in Local Storage	Low	OWASP ZAP + DevTools
F-06	Private IP address disclosure	Low	OWASP ZAP
F-07	Missing X-Content-Type-Options on specific responses	Low	OWASP ZAP

* Contextual finding — requires further validation before being presented as a confirmed, exploitable vulnerability.

9. Detailed Findings

9.1 F-01 — Content Security Policy (CSP) Header Not Set

F-01	Content Security Policy (CSP) Header Not Set	Medium
------	--	--------

Source	OWASP ZAP
ZAP Alert Reference	10038
CWE	693
WASC	15
Affected URL	http://127.0.0.1:3000/ (and other endpoints, including the login request and Socket.IO polling endpoint)

Observation

The application's responses did not include a Content-Security-Policy header. CSP provides an additional layer of protection against cross-site scripting and data-injection risks by allowing the application to declare which sources of scripts, styles, images, and other resources browsers are permitted to load or execute. No exploitation of this gap (such as a working cross-site scripting payload) was attempted or confirmed as part of this assessment.

Recommended Remediation

- Configure a restrictive Content-Security-Policy header appropriate to the application's legitimate scripts, styles, images, connections, and other resource needs.
- Where practical, begin with a Content-Security-Policy-Report-Only policy, tune it against real traffic, and then move to enforcement.

9.2 F-02 — Permissive Cross-Origin Resource Sharing (CORS)

F-02	Permissive Cross-Origin Resource Sharing (CORS)	Medium
------	---	--------

Source	OWASP ZAP + HTTP response header inspection
ZAP Alert Reference	10098 (Cross-Domain Misconfiguration)
CWE	264
WASC	14
Affected URL	http://127.0.0.1:3000/ (and static assets served with the same header)

Observation

The application's responses were observed to include the header `Access-Control-Allow-Origin: *`. This wildcard CORS configuration can allow cross-origin read requests from arbitrary third-party domains for resources that would otherwise be restricted by the browser's Same-Origin Policy. The practical security impact depends on whether sensitive resources are exposed through this configuration and whether authentication and authorization controls independently prevent unauthorized access. This assessment did not attempt to retrieve or exfiltrate authenticated data cross-origin, and no such exploitation is claimed.

Recommended Remediation

- Avoid wildcard CORS configuration where it is not strictly required.
- Restrict allowed origins to a trusted, explicit list of domains.
- Ensure sensitive APIs independently require appropriate authentication and authorization.
- Only expose resources cross-origin that genuinely need to be accessible in that manner.

9.3 F-03 — Missing Anti-Clickjacking Header on Socket.IO Polling Endpoint

F-03	Missing Anti-Clickjacking Header on Socket.IO Endpoint	Medium*
Source	OWASP ZAP + HTTP inspection	
ZAP Alert Reference	10020 (Missing Anti-clickjacking Header)	
CWE	1021	
WASC	15	
Affected Endpoint	http://127.0.0.1:3000/socket.io/?EIO=4&transport=polling&t=...&sid=...	

Observation

ZAP identified a response on the application's Socket.IO polling endpoint that did not include an anti-clickjacking header (either `X-Frame-Options` or a `Content-Security-Policy frame-ancestors` directive). This is an endpoint-specific observation rather than a site-wide one: as confirmed independently in Section 7.2, the main homepage response was observed to include `X-Frame-Options: SAMEORIGIN`. This finding should not be read as "the website has no clickjacking protection" — it applies specifically to the identified Socket.IO polling response. No clickjacking exploitation was attempted or demonstrated.

Recommended Remediation

- Ensure all browser-renderable responses — including auxiliary and real-time transport endpoints — include appropriate clickjacking protection, using `X-Frame-Options` and/or `Content-Security-Policy frame-ancestors`.

- Review whether the Socket.IO polling transport is expected to return browser-renderable content, and apply consistent header policy across the application's middleware.

9.4 F-04 — Session Identifier in Socket.IO URL

F-04	Session Identifier in Socket.IO URL	Medium*
------	-------------------------------------	---------

Source	OWASP ZAP
ZAP Alert Reference	3 (Session ID in URL Rewrite)
CWE	598
WASC	13
Affected Endpoint / Parameter	Socket.IO polling endpoint — parameter sid

Observation

ZAP flagged a sid parameter present in the Socket.IO polling and WebSocket-upgrade URLs as a session identifier exposed via URL rewriting. This sid value appears to be a Socket.IO transport-layer connection identifier rather than a confirmed application authentication-session identifier, and it has not been independently validated as such. This finding should therefore not be treated as a confirmed authentication-session vulnerability. Exposure of sensitive identifiers in URLs can, in general, create risk through browser history, server logs, referrer headers, or other URL-handling mechanisms, and the application context around this specific identifier merits further review.

Recommended Remediation

- Validate the purpose and sensitivity of the sid identifier used by the Socket.IO transport layer.
- Avoid placing genuinely sensitive authentication or session identifiers in URLs.
- Prefer secure, cookie-based session mechanisms for authentication state where appropriate.

9.5 F-05 — Authentication-Related Data in Browser Local Storage

F-05	Authentication-Related Data in Browser Local Storage	Low
------	--	-----

Source	OWASP ZAP + Firefox Developer Tools
ZAP Alert Reference	120001 (Sensitive Information in Browser localStorage)
CWE	359
WASC	13

Affected Page	http://127.0.0.1:3000/#/login?redirectUrl=%2Fbasket
----------------------	---

Observation

ZAP identified authentication-related data stored in the browser's Local Storage. This was independently confirmed using Firefox Developer Tools, which showed two Local Storage keys — email and token — populated for the application's origin (see Section 13, Figure 3). The actual email address and token/JWT value are not reproduced in this report and have been redacted from the supporting screenshot. JavaScript-accessible Local Storage can increase the exposure of authentication material if client-side script execution is ever compromised (for example, through an XSS vulnerability). No such compromise was demonstrated or claimed as part of this assessment; the observation is limited to the presence and location of the stored data.

Recommended Remediation

- Avoid storing sensitive authentication material in JavaScript-accessible Local Storage where safer mechanisms are available.
- Review the token-handling architecture end-to-end.
- Consider appropriately configured secure, HttpOnly cookies for authentication state.
- Where cookies are used for session/authentication purposes, apply appropriate security attributes (see Section 11).

9.6 F-06 — Private IP Address Disclosure

F-06	Private IP Address Disclosure	Low
-------------	--------------------------------------	------------

Source	OWASP ZAP
ZAP Alert Reference	2 (Private IP Disclosure)
CWE	497
WASC	13
Affected URL	http://127.0.0.1:3000/rest/admin/application-configuration

Observation

The application-configuration API response exposed private/internal network addresses (192.168.99.100:3000 and 192.168.99.100:4200). Private IP disclosure can provide reconnaissance information that may assist an attacker in understanding internal network layout. Because this instance is a locally hosted training deployment rather than a production system, this finding is treated as contextual security hardening rather than evidence of a high-impact production compromise.

Recommended Remediation

- Remove unnecessary internal/private IP addresses and infrastructure details from client-visible API responses.
- Review the /rest/admin/application-configuration endpoint for any additional internal configuration data that should not be exposed to unauthenticated clients.

9.7 F-07 — Missing X-Content-Type-Options Header on Specific Responses

F-07	Missing X-Content-Type-Options on Specific Responses	Low
------	--	-----

Source	OWASP ZAP
ZAP Alert Reference	10021 (X-Content-Type-Options Header Missing)
CWE	693
WASC	15
Affected Endpoint	Socket.IO polling endpoint (3 instances)

Observation

ZAP reported three response instances, all on the Socket.IO polling endpoint, missing the X-Content-Type-Options: nosniff header. As independently confirmed in Section 7.2, the main homepage response was observed to include X-Content-Type-Options: nosniff. This finding is therefore endpoint-specific rather than site-wide, and should not be read as "the website does not use X-Content-Type-Options."

Recommended Remediation

- Set X-Content-Type-Options: nosniff consistently across all applicable HTTP responses, including auxiliary transport endpoints such as Socket.IO polling.

10. Informational ZAP Alerts

In addition to the seven findings detailed in Section 9, OWASP ZAP reported five further alert categories at Informational risk level. These were reviewed and are presented here as contextual observations rather than confirmed security vulnerabilities.

Alert	ZAP Reference	Note
Authentication Request Identified	10111	ZAP recognized a login request pattern; informational only, confirms the application has a conventional authentication endpoint.
Information Disclosure — Information in Browser localStorage	120000	General (non-sensitive-flagged) data observed in Local Storage, in addition to the sensitive-flagged item covered as F-05.
Information Disclosure — Information in Browser sessionStorage	120000	General data observed in Session Storage; see Section 11 for the specific value identified.
Information Disclosure — Suspicious Comments	10027	Source code comments flagged by pattern matching; reviewed and not associated with any confirmed sensitive disclosure.
Modern Web Application	10109	ZAP's detection that the application is a modern single-page application (e.g., client-side routing); not a security weakness.

None of the five alerts above are counted as confirmed vulnerabilities in the risk summary in Section 8.

11. Browser Security Observations

11.1 Session Storage

Firefox Developer Tools showed a single Session Storage entry for the application's origin:

```
bid = 6
```

This value was reviewed and is not considered a significant security finding; it is not elevated to a vulnerability in this report.

11.2 Cookies

Firefox Developer Tools showed five cookies set under `http://127.0.0.1:3000`:

- language
- token
- welcomebanner_status

- cookieconsent_status
- continue_code

The token cookie's attributes were reviewed in detail (see Section 13, Figure 4). The actual cookie value is not reproduced in this report.

Attribute	Observed Value
Domain	127.0.0.1
HostOnly	true
HttpOnly	false
Secure	false
SameSite	None
Path	/
Size	745
Created	Sat, 05 Sep 2026 09:27:35 GMT
Expires / Max-Age	Sat, 05 Sep 2026 17:27:35 GMT

These attributes are presented as an additional browser-side observation, consistent with the client-side storage behavior discussed in F-05. In particular, HttpOnly: false means the cookie is readable by JavaScript running on the page (similar exposure to the Local Storage token noted in F-05), and SameSite: None with Secure: false is a weaker combination than is generally recommended for cookies carrying session-related values. No new numbered finding is introduced for this observation beyond what is already captured in F-05; it is documented here as supporting context.

12. Remediation Recommendations

The table below consolidates the recommended remediation for each of the seven findings in Section 9.

ID	Recommended Remediation
F-01	Implement a restrictive Content-Security-Policy header, tuned via Report-Only mode before enforcement.
F-02	Restrict CORS to a trusted, explicit list of origins; avoid wildcard access to sensitive resources.
F-03	Apply consistent anti-clickjacking headers (X-Frame-Options and/or CSP frame-ancestors) to all browser-renderable responses, including auxiliary transport endpoints.
F-04	Validate the purpose of the Socket.IO sid parameter and avoid exposing sensitive session identifiers in URLs.

ID	Recommended Remediation
F-05	Review browser token-storage architecture; prefer secure, HttpOnly cookies over JavaScript-accessible Local Storage for authentication material.
F-06	Remove unnecessary private/internal IP information from client-visible API responses.
F-07	Apply X-Content-Type-Options: nosniff consistently to all applicable HTTP responses.

13. Evidence

The figures below are drawn directly from the assessment evidence collected for this engagement. JWT/token values, session identifiers, and the account email address have been redacted from browser-storage screenshots prior to publication; all other details are unaltered.

Figure 1 — Nmap Service Scan Output

```
# Nmap 7.99 scan initiated Fri Sep  4 08:45:54 2026
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000084s latency).

PORT      STATE SERVICE VERSION
3000/tcp   open  ppp?
1 service unrecognized despite returning data.

Service detection performed.
# Nmap done at Fri Sep  4 08:46:05 2026 -- 1 IP address (1 host up) scanned in
11.30 seconds
```

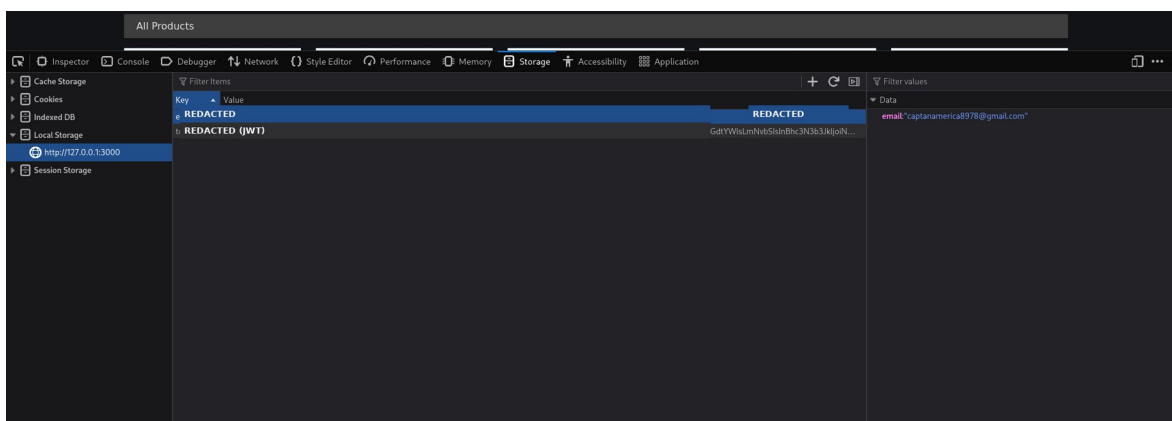
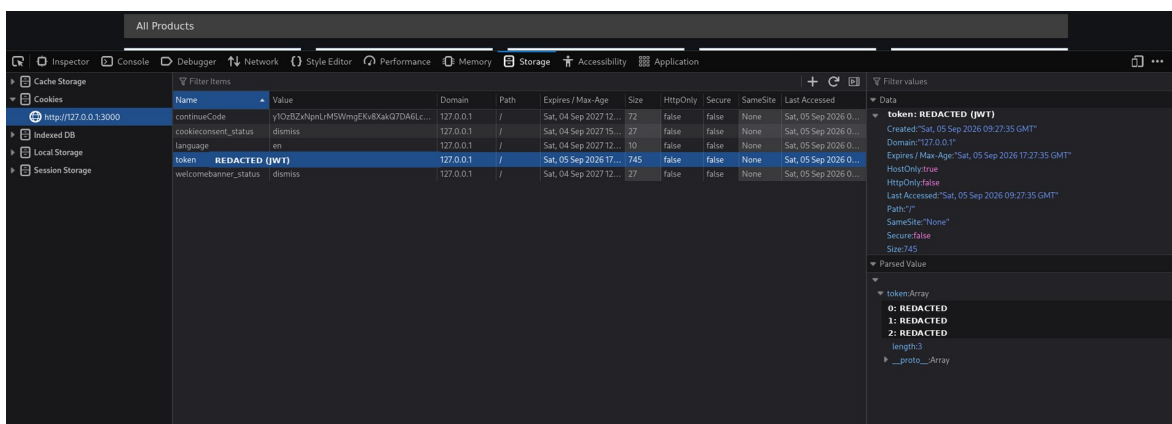
Figure 1 — Nmap service-version scan output for TCP port 3000 (evidence/nmap/01-basic-service-scan.txt).

Figure 2 — HTTP Response Headers (Firefox Developer Tools)

The screenshot shows the Firefox Developer Tools Network panel. The selected request is a POST to `/socket.io/46transport= polling&t=Q15Xl&sid=wKFPUpD5pUq`. The response headers are expanded, showing:

- Status: 200 OK
- Version: HTTP/1.1
- Transfered: 215 B (2 B size)
- Referer Policy: strict-origin-when-cross-origin
- DNS Resolution: System
- Response Headers (213 B):
 - Access-Control-Allow-Origin: `http://localhost:4200`
 - Connection: keep-alive
 - Content-Length: 2
 - Content-Type: text/html
 - Date: Fri, 04 Sep 2026 15:42:40 GMT
 - Keep-Alive: timeout=5
 - Vary: Origin
- Request Headers (659 B):
 - Accept: *
 - Accept-Encoding: gzip, deflate, br, zstd
 - Accept-Language: en-US;q=0.9

Figure 2 — Network panel showing HTTP response headers for the target application.

Figure 3 — Browser Local Storage*Figure 3 — Local Storage for http://127.0.0.1:3000, showing the email and token keys. Values redacted.***Figure 4 — Token Cookie Attributes***Figure 4 — Cookie attributes for the token cookie. Cookie value redacted.***Figure 5 — OWASP ZAP Passive Assessment Summary**

Metric	Value
Site	http://127.0.0.1:3000
ZAP Version	2.17.0
Generated	Friday, 4 September 2026
Observed Endpoints	49
High / Medium / Low / Informational	0 / 4 / 4 / 5

Figure 5 — Summary statistics extracted from the OWASP ZAP report (evidence/zap/zap-report.html, zap-alerts.json).

Full ZAP evidence files (zap-report.html and zap-alerts.json) and the Nmap terminal capture are retained in the accompanying project evidence directory (evidence/nmap/, evidence/zap/, evidence/browser-devtools/) and referenced by file name throughout this report.

14. Limitations

This assessment was passive, read-only, non-destructive, and conducted exclusively against a locally hosted training application. As a result, the findings in this report identify observed security weaknesses and configuration issues but do not establish exploitability for every finding. Specifically:

- F-03 is endpoint-specific: it applies to the Socket.IO polling response identified, not to the application's HTTP responses generally.
- F-04 requires further validation: the sid value appears to be a Socket.IO connection identifier rather than a confirmed application authentication-session identifier.
- The browser-storage observations underlying F-05 and Section 11 document the presence and location of client-side data; they do not, by themselves, prove that this data has been or could be exploited.
- F-06 (private IP disclosure) is contextual given the local training-environment setting and would need to be reassessed in a production network context.

This report does not claim, and should not be read to imply, that penetration testing or successful exploitation of any finding was performed.

15. Ethical Testing Statement

The assessment was performed against a locally hosted OWASP Juice Shop training application in a controlled and authorized environment. Testing was intentionally limited to passive, read-only, and non-destructive techniques. No login bypass, brute-force attack, denial-of-service activity, credential attack, or exploitation was performed as part of the intended assessment methodology.

16. Conclusion

This assessment identified several security-hardening opportunities in the locally hosted OWASP Juice Shop application. The most notable observations were the absence of a Content Security Policy header, a permissive CORS configuration, browser-side authentication-related storage exposure, private IP disclosure via an administrative API endpoint, and endpoint-specific gaps in security-header coverage.

The assessment demonstrates a practical vulnerability-assessment workflow combining Nmap service identification, OWASP ZAP passive analysis, direct HTTP header inspection, and Firefox Developer Tools browser-storage review — while deliberately avoiding exploitation and any activity that could cause harm to the target system. All findings are presented with their supporting source, affected scope, and recommended remediation, and are clearly distinguished from purely informational scanner output, in line with the passive and authorized nature of this engagement.